LAPORAN PRAKTIKUM PBO

PRAKTIKUM 5 Transaksi Dinamis (Polymorphism)

Disusun oleh :

M. Fajar Fadhilul Zikri

NIM: 2511533023

Dosen Pengampu: **Nurfiah, S.ST, M.Kom.**

Asisten Praktikum: **Muhammad Dawi Syauqi**



FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

DEPARTEMEN INFORMATIKA

TAHUN 2026

A. Tahapan Pelaksanaan Praktikum

Sebelumnya kita mengerjakan praktikum 5 ini kita terlebih dahulu mengerjakan praktikum 1,2,3 dan 4 berikut point-point dari laprak tsb:

Laprak 1: Di praktikum pertama kita belajar dasar OOP, yaitu cara memodelkan benda di dunia nyata ke dalam kode. Class itu ibarat cetak biru atau blueprint, dalam kasus kita namanya Rekening. Di dalamnya ada atribut (nomor rekening, nama pemilik, saldo) dan method (setor tunai, cek informasi). Object adalah hasil "cetakan" dari class itu yang dibuat pakai new, dan tiap object punya isi yang berdiri sendiri. Jadi rekening Mawar dan rekening Melati sama-sama dari class Rekening, tapi saldonya bisa beda dan tidak saling mengganggu. Constructor gunanya mengisi data awal otomatis saat object dibuat. Di bagian Main.java kita bikin menu CLI pakai Scanner dan while, supaya user bisa berinteraksi langsung dengan object-nya. Tugasnya menambah fitur tarik tunai (saldo tidak boleh minus, minimal 10.000) dan multi-akun pakai ArrayList supaya akun lama tidak tertimpa kalau buka rekening baru, plus menu ganti akun.

Laprak 2:penambahan class transaksi

```
public class Transaksi {
	private String idTransaksi;
	private String jenis;
	private double nominal;

	// construktor
	public Transaksi(String id, String jenis, double nominal) {
		this.idTransaksi = id;
		this.jenis = jenis;
		this.nominal = nominal;
	}

	public void cetakDetail() {
	  System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis +
" | Nominal: Rp" + nominal);
    }
```

Ini untuk tugas

```
ArrayList<Transaksi>riwayatTransaksi;
```

method setortunai:

```
public void setorTunai(double nominal) {
		if (nominal > 0) {
			saldo += nominal;

			String idTrx = "TRX-S-" +
System.currentTimeMillis();
			Transaksi trxBaru = new Transaksi(idTrx, "Kredit",
nominal);
```

```
                    riwayatTransaksi.add(trxBaru);

                    System.out.println("Setor tunai Rp" + nominal + "
Berhasil. Saldo saat ini: Rp" + saldo);
             } else {
                    System.out.println("Gagal: Nominal setor harus lebih
dari 0!");
             }
      }
public void tarikTunai(double nominal) {
             if (nominal < 10000) {

                    System.out.println("Transaksi Gagal: Minimal nominal
penarikan 10.000");
             } else if (nominal > saldo) {
                    System.out.println("Transaksi Gagal: Saldo tidak
mencukupi. Saldo Anda: Rp" + saldo);
             } else {
                    saldo -= nominal;
                    String idTrx = "TRX-T-" +
System.currentTimeMillis();
                    Transaksi trxBaru = new Transaksi(idTrx, "Debit",
nominal);
                    riwayatTransaksi.add(trxBaru);
                    System.out.println("Tarik tunai Rp" + nominal + "
Berhasil. Saldo saat ini: Rp" + saldo);
             }
      }
```

Class main:

```
case 6:
                           if(akunAktif == null) {
                                  System.out.println("Transaksi belum
ada!");
                           }else {
                                  akunAktif.cetakMutasi();
                           }
                           break;
```

Laprak 3:pada praktikum ini kita membahas tentang keamanan dan itegritas data berikut code yang berubah / berbeda dari Laprak 2:

1. Merubah variable menjadi private dan memberikan getter

```
public class Transaksi {
      private String idTransaksi;
      private String jenis;
      private double nominal;

      // construktor
      public Transaksi(String id, String jenis, double nominal) {
             this.idTransaksi = id;
             this.jenis = jenis;
             this.nominal = nominal;
      }
```

```
        // getter (Read-Only)
        public String getIdTransaksi() {return idTransaksi; }
        public String getJenis() { return jenis; }
        public double getNominal() { return nominal; }

        public void cetakDetail() {
          System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis +
" | Nominal: Rp" + nominal);
    }

}
```

2. Mengunci Entitas Rekening dan Menambahkan PIN pada rekening

```
public class Rekening {
    // 1. Mengunci atribut menjadi private
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin;  // Data sensitif
    private ArrayList<Transaksi> riwayatTransaksi;

    // 2. Modifikasi Constructor untuk menerima PIN
    public Rekening(String nomor, String nama, double saldoAwal, String
pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Validasi PIN saat Instansiasi / Constructor
        if (pinAwal != null && pinAwal.length() == 6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit!
Menggunakan PIN default 123456");
            this.pin = "123456";
        }

        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + "
berhasil dibuat dengan saldo Rp" + saldo);
    }

    // 3. Getter untuk atribut yang diizinkan dibaca publik
    public String getNomorRekening() {
        return nomorRekening;
    }

    public String getNamaPemilik() {
        return namaPemilik;
    }

    // 4. Method Otentikasi Internal PIN
    public boolean otentikasi(String inputPin) {
        return this.pin != null && this.pin.equals(inputPin);
    }
```

3.code untuk tugass 3

1. Buat rekening baru meminta user menginput pin 6 digit

```
case 1:
                        System.out.print("Masukkan No rekening: ");
                        String no = input.nextLine();
                        System.out.print("Masukkan Nama Pemilik: ");
                        String nama = input.nextLine();
                        System.out.print("Masukkan Saldo Awal: ");
                        double saldo = input.nextDouble();
                        input.nextLine();

                        //  Meminta input PIN 6 digit
                        System.out.print("Masukkan PIN (6 digit): ");
                        String pin = input.nextLine();


                        Rekening akunBaru = new Rekening(no, nama,
saldo, pin);
                        daftarRekening.add(akunBaru);

                        akunAktif = akunBaru;
                        break;
```

2.Memperbarui menu 3 tarik tunai

```
case 3:
                        if (akunAktif == null) {
                                System.out.println("Error: Mohon maaf,
anda belum memilih/memiliki rekening!");
                        } else {
                                // Minta PIN sebelum Tarik Tunai
                                System.out.print("Masukkan PIN Anda:
");
                                String inputPinTarik =
input.nextLine();

                                if
(akunAktif.otentikasi(inputPinTarik)) {
                                        System.out.print("Masukkan
nominal tarik: ");
                                        double tarik =
input.nextDouble();
                                        akunAktif.tarikTunai(tarik);
                                } else {
                                        System.out.println("Akses
Ditolak: PIN yang Anda masukkan salah!");
                                }
                        }
                        break;
```

3. Memperbarui menu 6 cetak mutase Pada praktikum ini, program sistem

```
        if (akunAktif == null) {
                                        System.out.println("Error: Anda belum
memilih/membuka rekening!");
                        } else {
                                // Minta PIN sebelum Cetak Mutasi
```

```
                                System.out.print("Masukkan PIN Anda:
");
                                String inputPinMutasi =
input.nextLine();

                                if
(akunAktif.otentikasi(inputPinMutasi)) {
                                    akunAktif.cetakMutasi();
                                } else {
                                    System.out.println("Akses
Ditolak: PIN yang Anda masukkan salah!");
                                }
                            }
                            break;
```

Laprak 4: Pada praktikum ini kita membahas tentang Ekspansi Produk Keuangan (Inheritance) berikut code yang berubah / berbeda dari Laprak 3:

Ubah privat ke protected:

```
protected ArrayList<Transaksi> riwayatTransaksi;
```

buat class RekeningTabungan dan RekeningGiro

untuk code tugas:

```
case 1:
                    System.out.print("Masukkan Nomor Rekening : ");
                    String nomor = input.nextLine();
                    System.out.print("Masukkan Nama Pemilik   : ");
                    String nama = input.nextLine();
                    System.out.print("Masukkan Saldo Awal     : ");
                    double saldoAwal = input.nextDouble();
                    input.nextLine();
                    System.out.print("Masukkan PIN  : ");
                    String pin = input.nextLine();

                    // Pilihan jenis produk rekening
                    System.out.println("Pilih Produk:");
                    System.out.println("1. Tabungan Umum");
                    System.out.println("2. Giro Bisnis");
                    System.out.print("Pilihan (1/2): ");
                    int jenisProduk = input.nextInt();
                    input.nextLine();

                    Rekening akunBaru = null;

                    if (jenisProduk == 1) {
                        // Input Suku Bunga untuk Rekening Tabungan
                        System.out.print("Masukkan Suku Bunga (%) : ");
                        double sukuBunga = input.nextDouble();
                        input.nextLine();

                        // Instansiasi objek RekeningTabungan
                        akunBaru = new RekeningTabungan(nomor, nama,
saldoAwal, pin, sukuBunga);
```

```
                        System.out.println("Rekening Tabungan berhasil
dibuat!");

                    } else if (jenisProduk == 2) {
                        // Input Batas Overdraft untuk Rekening Giro
                        System.out.print("Masukkan Batas Overdraft : ");
                        double batasOverdraft = input.nextDouble();
                        input.nextLine();

                        // Instansiasi objek RekeningGiro
                        akunBaru = new RekeningGiro(nomor, nama,
saldoAwal, pin, batasOverdraft);
                        System.out.println("Rekening Giro berhasil
dibuat!");

                    } else {
                        System.out.println("Pilihan produk tidak valid!
Pendaftaran dibatalkan.");
                        break;
                    }

                    // Upcasting: Menyimpan objek Subclass ke
daftarRekening bertipe Rekening
                    daftarRekening.add(akunBaru);
                    akunAktif = akunBaru;
                    break;
```

simulasi waktu akhir bulan:

```
case 7:
                    System.out.println("\n--- Simulasi Akhir Bulan ---");
                    if (akunAktif == null) {
                        System.out.println("Gagal: Silakan pilih / login
ke rekening terlebih dahulu!");
                        break;
                    }

                    // Pengecekan tipe objek menggunakan instanceof
                    if (akunAktif instanceof RekeningTabungan) {
                        // Downcasting eksplisit ke tipe RekeningTabungan
                        RekeningTabungan tabungan = (RekeningTabungan)
akunAktif;

                        // Memanggil method spesifik milik
RekeningTabungan
                        tabungan.tambahBungaAkhirBulan();
                    } else {
                        // Pesan penolakan jika akunAktif bukan
RekeningTabungan
                        System.out.println("Gagal: Fitur bunga akhir
bulan hanya berlaku untuk Rekening Tabungan.");
                    }
                    break;
```

Pada praktikum ini, program system perbankan mini dari Praktikum 4 dikembangkan dengan prinsip **Polimorfisme**. Polimorfisme memungkinkan satu

nama method dieksekusi dengan cara berbeda tergantung tipe objek aslinya saat program berjalan (runtime). Dua konsep utama yang dipraktikkan adalah *Method Overriding* dan *Dynamic Method Dispatch*, serta perancangan method yang menerima argumen polimorfik (objek dari berbagai subclass)

1. Membuat Class Rekening

Perubahan pada Rekening.java ada tiga: (1) atribut saldo dan riwayatTransaksi dipastikan protected agar dapat diakses subclass, (2) method tarikTunai diubah dari void menjadi boolean (true jika sukses, false jika gagal) agar hasilnya dapat diperiksa oleh method lain, dan (3) ditambahkan method transfer untuk tugas. Kode lengkapnya sebagai berikut.

```
public class Rekening {
    // 1. Mengunci atribut menjadi private
    private String nomorRekening;
    private String namaPemilik;
    protected double saldo;
    private String pin;  // Data sensitif
    protected ArrayList<Transaksi> riwayatTransaksi;

    // 2. Modifikasi Constructor untuk menerima PIN
    public Rekening(String nomor, String nama, double saldoAwal, String
pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Validasi PIN saat Instansiasi / Constructor
        if (pinAwal != null && pinAwal.length() == 6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit!
Menggunakan PIN default 123456");
            this.pin = "123456";
        }

        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + "
berhasil dibuat dengan saldo Rp" + saldo);
    }

    // 3. Getter untuk atribut yang diizinkan dibaca publik
    public String getNomorRekening() {
        return nomorRekening;
    }

    public String getNamaPemilik() {
        return namaPemilik;
    }

    // 4. Method Otentikasi Internal PIN
```

```java
    public boolean otentikasi(String inputPin) {
        return this.pin != null && this.pin.equals(inputPin);
    }


    public void setorTunai(double nominal) {
        if (nominal > 0) {
            this.saldo += nominal;

            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Setor tunai Rp" + nominal + " berhasil.
Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari
0!");
        }
    }

    public boolean tarikTunai(double nominal) { // diubah dari void ke
boolean
        if (nominal <= 0) {
            System.out.println("Gagal: Nominal tarik harus lebih dari
0!");
            return false;
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo tidak mencukupi.
Saldo Anda: Rp" + saldo);
            return false;
        } else {
            saldo -= nominal;

            String idTrx = "TRX-T-" + System.currentTimeMillis();
            riwayatTransaksi.add(new Transaksi(idTrx, "Debit",
nominal));

            System.out.println("Tarik tunai Rp" + nominal + " berhasil.
Sisa saldo: Rp" + saldo);
            return true; // Transaksi sukses
        }
    }

     // method transfer dengan parameter polimorfik
    public void transfer(Rekening tujuan, double nominal, String
pinInput) {
        // 1. Otentikasi
        if (!otentikasi(pinInput)) {
            System.out.println("Transfer dibatalkan: PIN salah.");
            return;
        }

        // Validasi tambahan: tidak boleh ke rekening sendiri
        if (tujuan == this) {
            System.out.println("Transfer dibatalkan: Rekening tujuan
tidak boleh sama dengan rekening asal.");
```

```
            return;
        }

        // 2. Tarik tunai (polimorfik: jika this adalah Giro, aturan
overdraft dipakai)
        boolean berhasil = this.tarikTunai(nominal);

        // 3. Jika penarikan sukses, setor ke rekening tujuan
        if (berhasil) {
            tujuan.setorTunai(nominal);
            System.out.println("Transfer Rp" + nominal + " ke rekening "
                    + tujuan.getNomorRekening() + " berhasil.");
        } else {
            System.out.println("Transfer gagal.");
        }
    }

    public void cetakMutasi() {
        System.out.println("\n=== MUTASI REKENING ===");
        System.out.println("NO. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("--------------------------------");

        if (riwayatTransaksi.isEmpty()) {
            System.out.println("Belum ada transaksi pada rekening ini");
        } else {
            for (Transaksi trx : riwayatTransaksi) {
                trx.cetakDetail();
            }
        }
    }

    public void cekInformasi() {
        System.out.println("--- INFO REKENING ---");
        System.out.println("NO. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir  : Rp" + saldo);
        System.out.println("--------------------");
    }
}
```

Penjelasan:

Jadi, di dalam class Rekening ini, kita bikin fondasi utamanya supaya data-data penting kayak PIN sama nomor rekening aman di dalam balutan enkapsulasi (private), jadi nggak bisa diakses sembarangan dari luar. Di sini kita juga nerapin konstruktor buat validasi awal supaya PIN wajib 6 digit. Nah, yang paling penting buat modul polimorfisme ini, method tarikTunai kita ubah tipe kembaliannya jadi boolean (bukan void lagi), supaya hasilnya bisa dilacak sukses atau gagalnya. Terus, kita juga nambahin method transfer yang nerima parameter polimorfik berupa objek Rekening (bisa dari Tabungan atau Giro). Alurnya itu simpel: pertama divalidasi dulu PIN-nya lewat otentikasi, terus kalau lolos, program bakal manggil

3. Membuat subclass RekeningTabungan:

Tidak ada perubahan sama seperti praktikum 4

```
public class RekeningTabungan extends Rekening {
    // Atribut spesifik yang hanya dimiliki oleh Tabungan
    private double sukuBunga;

    // Constructor Subclass
    public RekeningTabungan(String nomor, String nama, double saldoAwal,
String pinAwal, double sukuBunga) {

        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    // Method untuk menambahkan bunga pada akhir bulan
    public void tambahBungaAkhirBulan() {

        double nominalBunga = saldo * (sukuBunga / 100);
        saldo += nominalBunga;

        // Mencatat riwayat transaksi
        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga",
nominalBunga));

        System.out.println("Bunga " + sukuBunga + "% berhasil
ditambahkan: Rp " + nominalBunga);
    }
}
```

Class RekeningTabungan merupakan kelas anak yang mewarisi seluruh atribut dan metode dari kelas induk Rekening dengan menggunakan kata kunci extends. Kelas ini memiliki atribut spesifik sukuBunga serta konstruktor yang memanfaatkan pemanggilan super() untuk menginisialisasi data dasar rekening pada kelas induk. Di dalamnya juga terdapat metode tambahBungaAkhirBulan() yang berfungsi menghitung nominal bunga berdasarkan persentase sukuBunga, menambahkan nominal tersebut secara langsung ke variabel saldo, serta mencatat riwayat transaksi barunya ke dalam koleksi riwayatTransaksi.

3. Membuat Subclass RekeningGiro:

Pada Praktikum 4, RekeningGiro masih memakai logika tarikTunai milik Rekening, sehingga saldo tidak boleh minus. Pada praktikum ini, method tersebut ditimpa (*override*) agar Giro boleh menarik dana hingga saldo + batasOverdraft.

```
public class RekeningGiro extends Rekening {
    private double batasOverdraft;
```

```
    public RekeningGiro(String nomor, String nama, double saldoAwal,
String pinAwal, double batasOverdraft) {
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    // IMPLEMENTASI POLIMORFISME (OVERRIDING)
    @Override
    public boolean tarikTunai(double nominal) {
        if (nominal <= 0) {
            System.out.println("Gagal: Nominal tarik Giro harus lebih
dari 0!");
            return false;
        } else if (nominal > (saldo + batasOverdraft)) {
            System.out.println("Transaksi Gagal: Melebihi batas
overdraft! Maksimal penarikan: Rp" + (saldo + batasOverdraft));
            return false;
        } else {
            saldo -= nominal; // Saldo bisa bernilai negatif di skema
peminjaman overdraft

            String idTrx = "TRX-G-" + System.currentTimeMillis();
            riwayatTransaksi.add(new Transaksi(idTrx, "Debit Giro",
nominal));

            System.out.println("Tarik tunai Giro Rp" + nominal + "
berhasil. Saldo saat ini: Rp" + saldo);
            return true;
        }
    }

    public double getBatasOverdraft() {
        return batasOverdraft;
    }
}
```

untuk class RekeningGiro ini merupakan sub-kelas dari Rekening yang dirancang khusus buat nerapin konsep polimorfisme lewat *method overriding*. Di sini kita nambahin atribut baru khusus giro yaitu batasOverdraft. Nah, keunggulan utamanya ada di method tarikTunai yang sudah dioverride: kalau di rekening biasa saldo nggak boleh minus, di giro ini salidonya dibolehin sampai batas negatif tertentu sesuai ketentuan *overdraft* yang diberikan. Jadi waktu nasabah mau tarik tunai, program bakal ngecek apakah nominalnya masih dalam batas aman saldo ditambah batasOverdraft. Kalau lolos, saldo otomatis berkurang jadi minus, riwayat transaksinya otomatis tercatat sebagai debit giro, dan proses transaksinya sukses mengembalikan nilai true.

4. Membuat Class Main

Pada Main1.java, kode menu 3 (Tarik Tunai) tidak diubah logikanya. Perubahan hanya penambahan input.nextLine() setelah nextDouble() agar buffer bersih, serta penambahan menu 8 (Transfer ke Rekening Lain).

```
public class Main1 {
	private static Rekening akunAktif;

	public static void main(String[] args) {
		Scanner input = new Scanner(System.in);

		ArrayList<Rekening> daftarRekening = new ArrayList<>();
		akunAktif = null;
		boolean isRunning = true;

		System.out.println("=== SISTEM PERBANKAN MINI ===");

		while (isRunning) {
			System.out.println("\n-----------------------------
");
			if (akunAktif != null) {

				System.out.println("Akun Aktif: " +
akunAktif.getNamaPemilik() + " (" + akunAktif.getNomorRekening() + ")");
			} else {
				System.out.println("Akun Aktif: Belum Ada");
			}
			System.out.println("-----------------------------");
			System.out.println("Menu Utama:");
			System.out.println("1. Buka Rekening Baru");
			System.out.println("2. Setor Tunai");
			System.out.println("3. Tarik Tunai");
			System.out.println("4. Cek Informasi Rekening");
			System.out.println("5. Ganti Akun");
			System.out.println("6. Cetak Mutasi (Riwayat)");
			System.out.println("7. Simulasi Akhir Bulan (Khusus
Tabungan)");
			System.out.println("0. Keluar");
			System.out.print("Pilih Menu: ");
			int pilihan = input.nextInt();
			input.nextLine();

			switch (pilihan) {
			case 1:
			System.out.print("Masukkan Nomor Rekening : ");
			String nomor = input.nextLine();
			System.out.print("Masukkan Nama Pemilik   : ");
			String nama = input.nextLine();
			System.out.print("Masukkan Saldo Awal     : ");
			double saldoAwal = input.nextDouble();
			input.nextLine();
			System.out.print("Masukkan PIN  : ");
			String pin = input.nextLine();

			// Pilihan jenis produk rekening
```

```
                    System.out.println("Pilih Produk:");
                    System.out.println("1. Tabungan Umum");
                    System.out.println("2. Giro Bisnis");
                    System.out.print("Pilihan (1/2): ");
                    int jenisProduk = input.nextInt();
                    input.nextLine();

                    Rekening akunBaru = null;

                    if (jenisProduk == 1) {
                        // Input Suku Bunga untuk Rekening Tabungan
                        System.out.print("Masukkan Suku Bunga (%) : ");
                        double sukuBunga = input.nextDouble();
                        input.nextLine();

                        // Instansiasi objek RekeningTabungan
                        akunBaru = new RekeningTabungan(nomor, nama,
saldoAwal, pin, sukuBunga);
                        System.out.println("Rekening Tabungan berhasil
dibuat!");

                    } else if (jenisProduk == 2) {
                        // Input Batas Overdraft untuk Rekening Giro
                        System.out.print("Masukkan Batas Overdraft : ");
                        double batasOverdraft = input.nextDouble();
                        input.nextLine();

                        // Instansiasi objek RekeningGiro
                        akunBaru = new RekeningGiro(nomor, nama,
saldoAwal, pin, batasOverdraft);
                        System.out.println("Rekening Giro berhasil
dibuat!");

                    } else {
                        System.out.println("Pilihan produk tidak valid!
Pendaftaran dibatalkan.");
                        break;
                    }

                    // Upcasting: Menyimpan objek Subclass ke
daftarRekening bertipe Rekening
                    daftarRekening.add(akunBaru);
                    akunAktif = akunBaru;
                    break;

                     case 2:
                           if (akunAktif == null) {
                                 System.out.println("Error: Mohon maaf,
anda belum memilih/memiliki rekening!");
                           } else {
                                 System.out.print("Masukkan nominal
setor: ");
                                 double setor = input.nextDouble();
                                 akunAktif.setorTunai(setor);
                           }
                           break;
```

```
                    case 3:
                        if (akunAktif == null) {
                            System.out.println("Error: Mohon maaf,
anda belum memilih/memiliki rekening!");
                        } else {
                            // Minta PIN sebelum Tarik Tunai
                            System.out.print("Masukkan PIN Anda:
");
                            String inputPinTarik =
input.nextLine();

                            if
(akunAktif.otentikasi(inputPinTarik)) {
                                System.out.print("Masukkan
nominal tarik: ");
                                double tarik =
input.nextDouble();
                                akunAktif.tarikTunai(tarik);
                            } else {
                                System.out.println("Akses
Ditolak: PIN yang Anda masukkan salah!");
                            }
                        }
                        break;

                    case 4:
                        if (akunAktif == null) {
                            System.out.println("Error: Anda belum
memilih/membuka rekening!");
                        } else {
                            akunAktif.cekInformasi();
                        }
                        break;

                    case 5:
                        if (daftarRekening.isEmpty()) {
                            System.out.println("Belum ada rekening
yang terdaftar di dalam sistem.");
                        } else {
                            System.out.print("Masukkan No Rekening
yang dicari: ");
                            String noCari = input.nextLine();
                            boolean ditemukan = false;

                            for (Rekening rek : daftarRekening) {
                                if
(rek.getNomorRekening().equalsIgnoreCase(noCari)) {
                                    akunAktif = rek;
                                    ditemukan = true;
                                    // Menggunakan getter
getNamaPemilik()

      System.out.println("Berhasil beralih ke rekening atas nama: " +
akunAktif.getNamaPemilik());
                                    break;
                                }
                            }
```

```
                                if (!ditemukan) {
                                        System.out.println("Error: Nomor
rekening tidak ditemukan!");
                                }
                            }
                            break;

                        case 6:
                            if (akunAktif == null) {
                                System.out.println("Error: Anda belum
memilih/membuka rekening!");
                            } else {
                                // Minta PIN sebelum Cetak Mutasi
                                System.out.print("Masukkan PIN Anda:
");
                                String inputPinMutasi =
input.nextLine();

                                if
(akunAktif.otentikasi(inputPinMutasi)) {
                                    akunAktif.cetakMutasi();
                                } else {
                                    System.out.println("Akses
Ditolak: PIN yang Anda masukkan salah!");
                                }
                            }
                            break;

                        case 7:
                        System.out.println("\n--- Simulasi Akhir Bulan ---");
                        if (akunAktif == null) {
                            System.out.println("Gagal: Silakan pilih / login
ke rekening terlebih dahulu!");
                            break;
                        }

                        // Pengecekan tipe objek menggunakan instanceof
                        if (akunAktif instanceof RekeningTabungan) {
                            // Downcasting eksplisit ke tipe RekeningTabungan
                            RekeningTabungan tabungan = (RekeningTabungan)
akunAktif;

                            // Memanggil method spesifik milik
RekeningTabungan
                            tabungan.tambahBungaAkhirBulan();
                        } else {
                            // Pesan penolakan jika akunAktif bukan
RekeningTabungan
                            System.out.println("Gagal: Fitur bunga akhir
bulan hanya berlaku untuk Rekening Tabungan.");
                        }
                        break;

                        case 8:
                    if (akunAktif == null) {
```

```
                    System.out.println("Error: Anda belum
memilih/membuka rekening!");
                    break;
                }
                System.out.print("Masukkan Nomor Rekening Tujuan : ");
                String noTujuan = input.nextLine();

                Rekening akunTujuan = null;
                for (Rekening rek : daftarRekening) {
                    if
(rek.getNomorRekening().equalsIgnoreCase(noTujuan)) {
                        akunTujuan = rek;
                        break;
                    }
                }

                if (akunTujuan == null) {
                    System.out.println("Error: Rekening tujuan tidak
ditemukan!");
                } else {
                    System.out.print("Masukkan nominal transfer : ");
                    double nominalTransfer = input.nextDouble();
                    input.nextLine();
                    System.out.print("Masukkan PIN Anda : ");
                    String pinTransfer = input.nextLine();

                    akunAktif.transfer(akunTujuan, nominalTransfer,
pinTransfer);
                }
                break;


                    case 0:
                          isRunning = false;
                          System.out.println("Sistem ditutup. Terima
kasih!");
                          break;

                    default:
                          System.out.println("Pilihan tidak valid!");
                    }
            }
            input.close();
      }
}
```

Penjelasan: di class Main1 ini kita bikin program utama berupa sistem CLI (Command Line Interface) buat menjalankan aplikasi perbankannya secara interaktif. Di sini kita nyiapin *looping* menu utama pakai while yang nampilin berbagai pilihan fitur, mulai dari buka rekening baru (yang bisa milih antara tabungan umum atau giro), setor tunai, tarik tunai, cek informasi, ganti akun, cetak mutasi, sampai simulasi akhir bulan khusus tabungan. Selain itu, kodenya juga

sudah menerapin konsep *upcasting* pas masukin objek anak ke daftarRekening, terus ada juga pengecekan tipe data pakai instanceof sama *downcasting* pas mau manggil fitur khusus bunga akhir bulan. Tiap kali mau ngelakuin transaksi sensitif kayak tarik tunai atau cetak mutasi, sistemnya juga udah otomatis ngecek keamanan lewat autentikasi PIN si akun aktif.

5. Membuat Class Transaksi

seperti parktikum 4 tidak ada perubahan

```
public class Transaksi {
	private String idTransaksi;
	private String jenis;
	private double nominal;

	// construktor
	public Transaksi(String id, String jenis, double nominal) {
		this.idTransaksi = id;
		this.jenis = jenis;
		this.nominal = nominal;
	}

	// getter (Read-Only)
	public String getIdTransaksi() {return idTransaksi; }
	public String getJenis() { return jenis; }
	public double getNominal() { return nominal; }

	public void cetakDetail() {
	  System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis +
" | Nominal: Rp" + nominal);
    }

}
```

Penjelasan: Class Transaksi berfungsi sebagai cetak biru (blueprint) untuk memodelkan data entitas riwayat transaksi pada sistem perbankan. Class ini mengamankan tiga atribut utamanya, yaitu idTransaksi, jenis, dan nominal dengan visibilitas *private*. Constructor digunakan untuk menginisialisasi nilai awal seluruh atribut saat transaksi baru dibuat, sedangkan beberapa method *getter* disediakan sebagai akses *read-only* untuk membaca data tanpa dapat mengubahnya. Selain itu, terdapat method cetakDetail() yang berfungsi untuk menampilkan rincian informasi transaksi secara rapi ke konsol.

B. Latihan/Tugas

Sekarang sistem telah memiliki berbagai bentuk rekening. Silahkan memperbarui antarmuka CLI di Main.java agar mendukung produk-produk baru ini secara interaktif.

Tugas: Fitur Transfer Antar Rekening

1. Buat Method Transfer : Di dalam class Rekening.java, tambahkan satu method bar dengan nama : public void transfer(Rekening tujuan, double nominal, String pinInput) • Method ini menerima parameter polimorfik Rekening tujuan, bisa menerima Tabungan atau Giro. • Logika Method ▪ Pertama, lakukan otentikasi(pinInput). Jika false, batalkan. MODUL PRAKTIKUM PBO - JAVA NURFIAH ▪ Kedua, panggil this.tarikTunai(nominal). (overriding, jika this adalah Giro, ia otomatis menggunakan aturan overdraft). ▪ Ketiga, periksa kembalian nilai tarikTunai. Jika true, panggil tujuan.setorTunai(nominal).

2. Integrasi ke CLI Utama: Buka Main.java dan tambahkan menu 8. Transfer ke Rekening Lain. • Minta pengguna memasukkan Nomor Rekening Tujuan. • Cari nomor tersebut di dalam daftarRekening. • Jika ketemu, minta nominal transfer akunAktif.transfer(akunTujuan, nominal, pin).

1. Tugas 1 Fitur Transfer Antar Rekening

```
// method transfer dengan parameter polimorfik
    public void transfer(Rekening tujuan, double nominal, String
pinInput) {
        // 1. Otentikasi
        if (!otentikasi(pinInput)) {
            System.out.println("Transfer dibatalkan: PIN salah.");
            return;
        }

        // Validasi tambahan: tidak boleh ke rekening sendiri
        if (tujuan == this) {
            System.out.println("Transfer dibatalkan: Rekening tujuan
tidak boleh sama dengan rekening asal.");
            return;
        }

        // 2. Tarik tunai (polimorfik: jika this adalah Giro, aturan
overdraft dipakai)
        boolean berhasil = this.tarikTunai(nominal);

        // 3. Jika penarikan sukses, setor ke rekening tujuan
        if (berhasil) {
            tujuan.setorTunai(nominal);
```

```
            System.out.println("Transfer Rp" + nominal + " ke rekening "
                    + tujuan.getNomorRekening() + " berhasil.");
        } else {
            System.out.println("Transfer gagal.");
        }
    }
```

Penjelasan: di dalam method transfer ini kita menerapkan parameter polimorfik bertipe Rekening (tujuan), yang artinya method ini bisa menerima objek apa saja yang merupakan turunan dari Rekening, baik itu Tabungan maupun Giro. Alurnya dimulai dengan validasi PIN milik pengirim lewat method otentikasi(), dilanjut dengan pengecekan supaya tidak bisa transfer ke rekening sendiri. Setelah itu, program melakukan penarikan saldo lewat this.tarikTunai(nominal) yang sifatnya polimorfik—jadi kalau yang ngirim itu akun RekeningGiro, otomatis aturan *overdraft*-nya langsung terpakai. Terakhir, kalau proses penarikannya berhasil (true), saldo di rekening tujuan akan langsung bertambah lewat method tujuan.setorTunai(nominal).

2. Tugas 2 Integrasi ke CLI Utama: Buka Main.java dan tambahkan menu 8. Transfer ke Rekening Lain.

```
case 8:
                if (akunAktif == null) {
                    System.out.println("Error: Anda belum
memilih/membuka rekening!");
                    break;
                }
                System.out.print("Masukkan Nomor Rekening Tujuan : ");
                String noTujuan = input.nextLine();

                Rekening akunTujuan = null;
                for (Rekening rek : daftarRekening) {
                    if
(rek.getNomorRekening().equalsIgnoreCase(noTujuan)) {
                        akunTujuan = rek;
                        break;
                    }
                }

                if (akunTujuan == null) {
                    System.out.println("Error: Rekening tujuan tidak
ditemukan!");
                } else {
```

```
                    System.out.print("Masukkan nominal transfer : ");
                    double nominalTransfer = input.nextDouble();
                    input.nextLine();
                    System.out.print("Masukkan PIN Anda : ");
                    String pinTransfer = input.nextLine();

                    akunAktif.transfer(akunTujuan, nominalTransfer,
pinTransfer);
                }
                break;
```

Penjelasan: untuk kode case 8 ini kita nambahin menu baru sesuai tugas modul buat fitur transfer antar rekening. Alurnya itu pertama dicek dulu apakah sudah ada akun yang aktif. Kalau ada, sistem bakal minta pengguna buat memasukkan nomor rekening tujuan, terus ngelewatin proses *searching* di dalam daftarRekening buat nyari objek rekening tujuannya. Kalau rekeningnya ketemu, program bakal lanjut nanyain nominal uang yang mau ditransfer sama minta konfirmasi PIN buat keamanan. Terus di bagian akhir, program tinggal manggil method akunAktif.transfer(akunTujuan, nominalTransfer, pinTransfer), di mana nanti sistem bakal otomatis ngejalanin validasi PIN, penarikan saldo secara polimorfik (yang otomatis nyesuaiin aturan giro atau tabungan), sampai penyetoran dana ke rekening tujuan kalau transaksinya sukses.

6. Hasil Output Program

```
=== SISTEM PERBANKAN MINI ===

----------------------------
Akun Aktif: Belum Ada
----------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
0. Keluar
Pilih Menu: 1
Masukkan Nomor Rekening : 12333
Masukkan Nama Pemilik   : jar
Masukkan Saldo Awal     : 1000000
Masukkan PIN  : 123456
Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
Pilihan (1/2): 1
```

```
Masukkan Suku Bunga (%) : 5
Rekening atas nama jar berhasil dibuat dengan saldo Rp1000000.0
Rekening Tabungan berhasil dibuat!

----------------------------
Akun Aktif: jar (12333)
----------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
0. Keluar
Pilih Menu: 1
Masukkan Nomor Rekening : 23444
Masukkan Nama Pemilik   : iyan
Masukkan Saldo Awal     : 100000
Masukkan PIN  : 098765
Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
Pilihan (1/2): 2
Masukkan Batas Overdraft : 500000
Rekening atas nama iyan berhasil dibuat dengan saldo Rp100000.0
Rekening Giro berhasil dibuat!

----------------------------
Akun Aktif: iyan (23444)
----------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
0. Keluar
Pilih Menu: 3
Masukkan PIN Anda: 098765
Masukkan nominal tarik: 400000
Tarik tunai Giro Rp400000.0 berhasil. Saldo saat ini: Rp-300000.0

----------------------------
Akun Aktif: iyan (23444)
----------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
```

```
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
0. Keluar
Pilih Menu: 3
Masukkan PIN Anda: 098765
Masukkan nominal tarik: 300000
Transaksi Gagal: Melebihi batas overdraft! Maksimal penarikan:
Rp200000.0

----------------------------
Akun Aktif: iyan (23444)
----------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
0. Keluar
Pilih Menu: 5
Masukkan No Rekening yang dicari: 12333
Berhasil beralih ke rekening atas nama: jar

----------------------------
Akun Aktif: jar (12333)
----------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
0. Keluar
Pilih Menu: 8
Masukkan Nomor Rekening Tujuan : 23444
Masukkan nominal transfer : 200000
Masukkan PIN Anda : 123456
Tarik tunai Rp200000.0 berhasil. Sisa saldo: Rp800000.0
Setor tunai Rp200000.0 berhasil. Saldo saat ini: Rp-100000.0
Transfer Rp200000.0 ke rekening 23444 berhasil.

----------------------------
Akun Aktif: jar (12333)
----------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
```

```
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
0. Keluar
Pilih Menu: 5
Masukkan No Rekening yang dicari: 23444
Berhasil beralih ke rekening atas nama: iyan

----------------------------
Akun Aktif: iyan (23444)
----------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
0. Keluar
Pilih Menu: 8
Masukkan Nomor Rekening Tujuan : 12333
Masukkan nominal transfer : 300000
Masukkan PIN Anda : 098765
Tarik tunai Giro Rp300000.0 berhasil. Saldo saat ini: Rp-400000.0
Setor tunai Rp300000.0 berhasil. Saldo saat ini: Rp1100000.0
Transfer Rp300000.0 ke rekening 12333 berhasil.

----------------------------
Akun Aktif: iyan (23444)
----------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
0. Keluar
Pilih Menu: 8
Masukkan Nomor Rekening Tujuan : 12333
Masukkan nominal transfer : 250000
Masukkan PIN Anda : 567890
Transfer dibatalkan: PIN salah.

----------------------------
Akun Aktif: iyan (23444)
----------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
```

```
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
0. Keluar
Pilih Menu: 8
Masukkan Nomor Rekening Tujuan : 23753
Error: Rekening tujuan tidak ditemukan!

----------------------------
Akun Aktif: iyan (23444)
----------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
0. Keluar
Pilih Menu: 5
Masukkan No Rekening yang dicari: 12333
Berhasil beralih ke rekening atas nama: jar

----------------------------
Akun Aktif: jar (12333)
----------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
0. Keluar
Pilih Menu: 8
Masukkan Nomor Rekening Tujuan : 23444
Masukkan nominal transfer : 5000000
Masukkan PIN Anda : 123456
Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp1100000.0
Transfer gagal.

----------------------------
Akun Aktif: jar (12333)
----------------------------
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer
```

```
0. Keluar
Pilih Menu: 0
Sistem ditutup. Terima kasih!
```

Penjelasan: hasil output menunjukkan bahwa fitur transfer berhasil kita terapkan Dimana kita dapat melakukan transfer dari rekening Tabungan ke giro dan sebaliknya dan jika saldo tidak mencukupi ada notifikasinya lalu untuk fitur Uji Dynamic Dispatch juga berhasil Dimana jika saldo yang di Tarik melebihi saldo saat ini selama tidak melewati overdraft maka bisa di Tarik dan saldo menjadi mines.

Kesimpulan:

Dari praktikum 5 ini dapat disimpulkan bahwa polimorfisme membuat program perbankan menjadi lebih fleksibel dan mudah dikembangkan. Dengan method overriding, RekeningGiro dapat menimpa method tarikTunai() milik Rekening sehingga aturan penarikannya berbeda, yaitu boleh melewati saldo sampai batas overdraft, dan anotasi @Override membantu compiler mendeteksi kesalahan penulisan method. Berkat dynamic method dispatch, JVM memilih versi tarikTunai yang sesuai saat program berjalan berdasarkan objek aslinya, sehingga kode di Main1 tidak perlu diubah dan tidak perlu if-else untuk mengecek jenis rekening. Method transfer(Rekening tujuan, ...) juga menerima argumen polimorfik, jadi satu method dapat dipakai untuk rekening Tabungan maupun Giro dan tetap siap jika nanti ada jenis rekening baru. Terakhir, perubahan return type tarikTunai menjadi boolean penting agar transfer hanya menyetor ke rekening tujuan setelah penarikan benar-benar berhasil, sehingga saldo tetap konsisten.

Link Video: https://youtu.be/eZ8EN_GhNRA